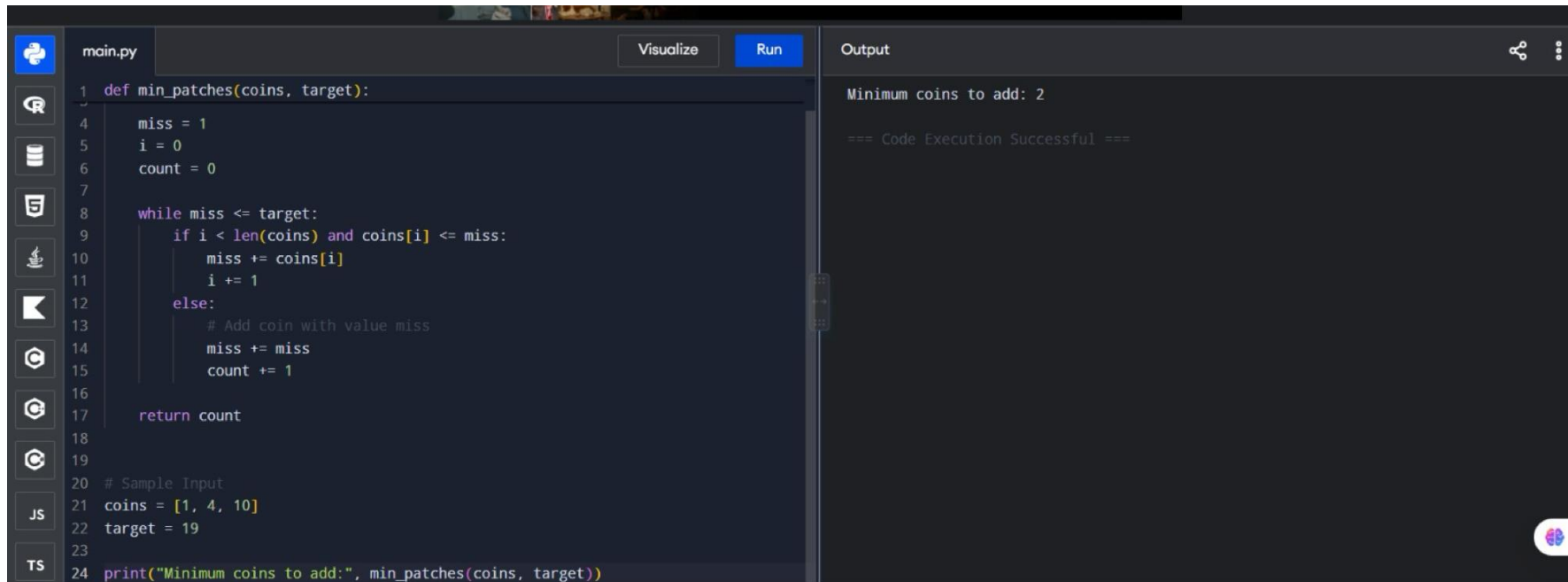


1. Maximum Coins You Can Get



The image shows a code editor interface with a dark theme. On the left, a sidebar contains icons for Python, JavaScript, and TypeScript. The main editor area displays a Python file named `main.py`. The code defines a function `min_patches` that takes a list of coin values and a target value, and returns the minimum number of coins needed to reach the target. The function uses a greedy algorithm. Below the function, sample input is provided: `coins = [1, 4, 10]` and `target = 19`. The script concludes with a `print` statement. On the right, an 'Output' panel shows the result of the execution: 'Minimum coins to add: 2' followed by a success message. The interface includes 'Visualize' and 'Run' buttons at the top of the editor.

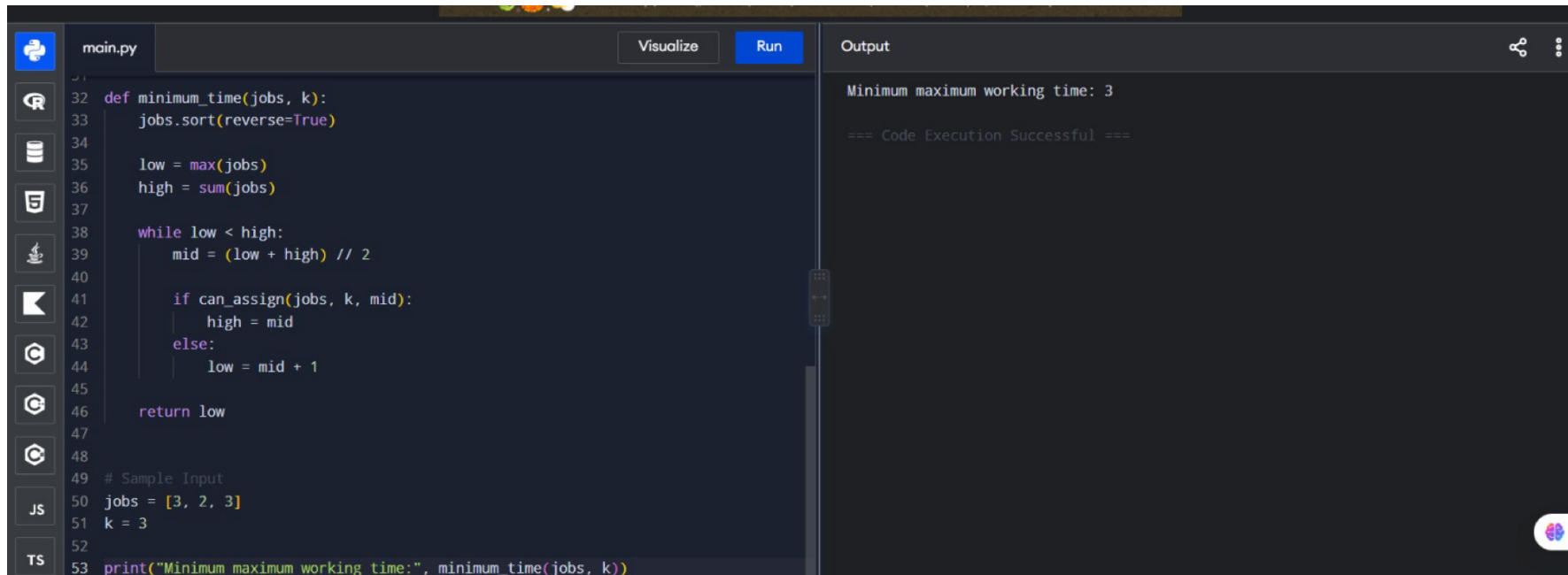
```
1 def min_patches(coins, target):
2
3     miss = 1
4     i = 0
5     count = 0
6
7     while miss <= target:
8         if i < len(coins) and coins[i] <= miss:
9             miss += coins[i]
10            i += 1
11        else:
12            # Add coin with value miss
13            miss += miss
14            count += 1
15
16    return count
17
18
19
20 # Sample Input
21 coins = [1, 4, 10]
22 target = 19
23
24 print("Minimum coins to add:", min_patches(coins, target))
```

Output

Minimum coins to add: 2

=== Code Execution Successful ===

2. Minimum Number of Coins to Add



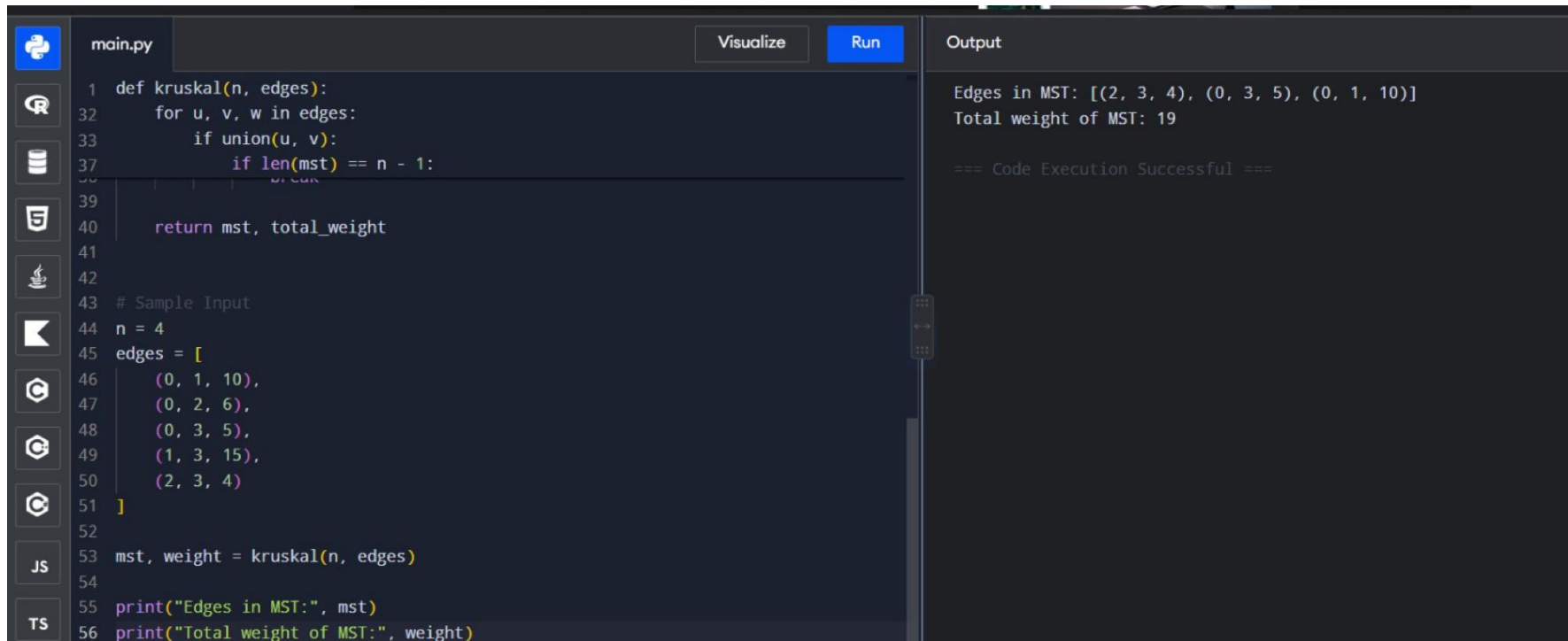
```
main.py Visualize Run
32 def minimum_time(jobs, k):
33     jobs.sort(reverse=True)
34
35     low = max(jobs)
36     high = sum(jobs)
37
38     while low < high:
39         mid = (low + high) // 2
40
41         if can_assign(jobs, k, mid):
42             high = mid
43         else:
44             low = mid + 1
45
46     return low
47
48 # Sample Input
49 jobs = [3, 2, 3]
50 k = 3
51
52
53 print("Minimum maximum working time:", minimum_time(jobs, k))
```

Output

Minimum maximum working time: 3

=== Code Execution Successful ===

3. Minimum Maximum Working Time



The screenshot displays a Jupyter Notebook environment with a dark theme. The left sidebar contains icons for various tools: Python, a search icon, a file explorer, a terminal, a Jupyter logo, a flag, a notebook icon, a console icon, a settings icon, a JS icon, and a TS icon. The main area shows a code editor with a file named 'main.py'. The code implements Kruskal's algorithm to find the Minimum Spanning Tree (MST) of a graph with 4 nodes and 6 edges. The edges are: (0, 1, 10), (0, 2, 6), (0, 3, 5), (1, 3, 15), (2, 3, 4), and (2, 3, 4). The algorithm returns the MST edges and the total weight. The output panel on the right shows the results of the execution.

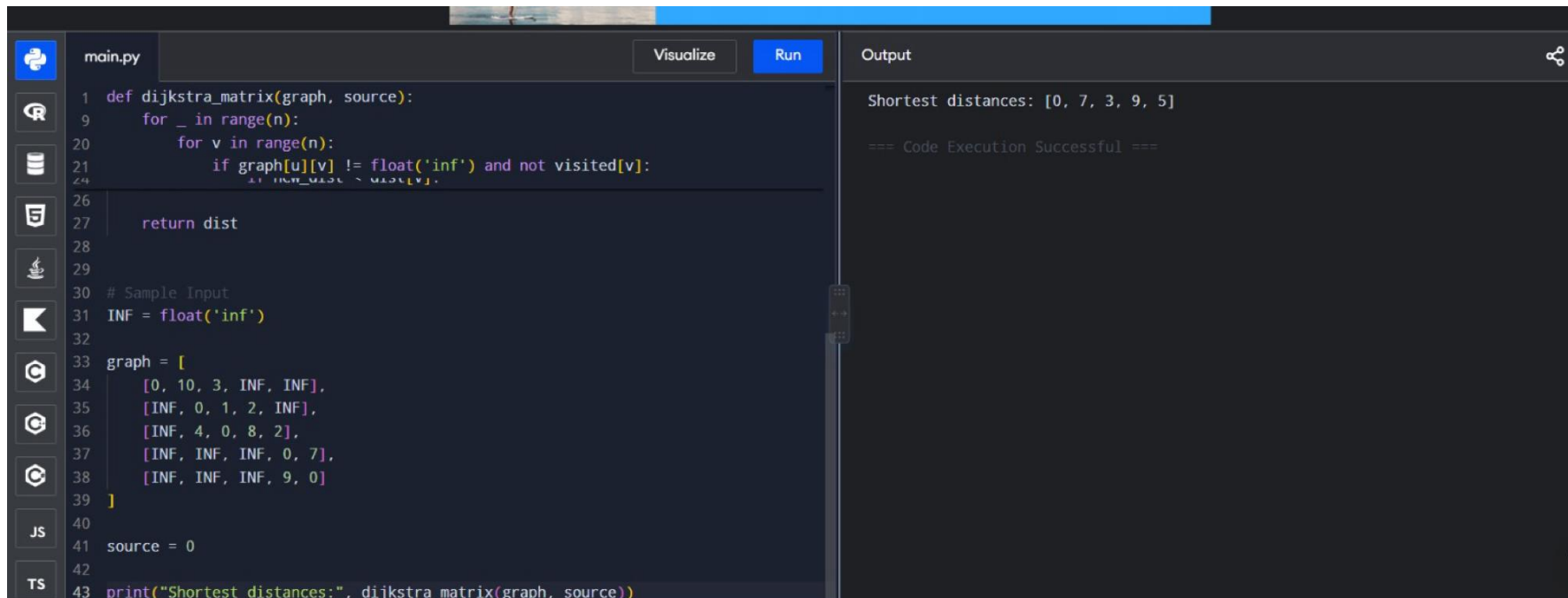
```
main.py Visualize Run
1 def kruskal(n, edges):
32     for u, v, w in edges:
33         if union(u, v):
37             if len(mst) == n - 1:
39                 break
40     return mst, total_weight
41
42
43 # Sample Input
44 n = 4
45 edges = [
46     (0, 1, 10),
47     (0, 2, 6),
48     (0, 3, 5),
49     (1, 3, 15),
50     (2, 3, 4),
51 ]
52
53 mst, weight = kruskal(n, edges)
54
55 print("Edges in MST:", mst)
56 print("Total weight of MST:", weight)
```

Output

```
Edges in MST: [(2, 3, 4), (0, 3, 5), (0, 1, 10)]
Total weight of MST: 19

=== Code Execution Successful ===
```

4. Maximum Profit in Job Scheduling



The screenshot shows a code editor with a file named `main.py`. The code implements a function `dijkstra_matrix` that calculates the shortest distances from a source node to all other nodes in a graph. The graph is represented as an adjacency list where each element is a list of [neighbor, weight]. The source node is 0. The output of the function is a list of shortest distances: [0, 7, 3, 9, 5].

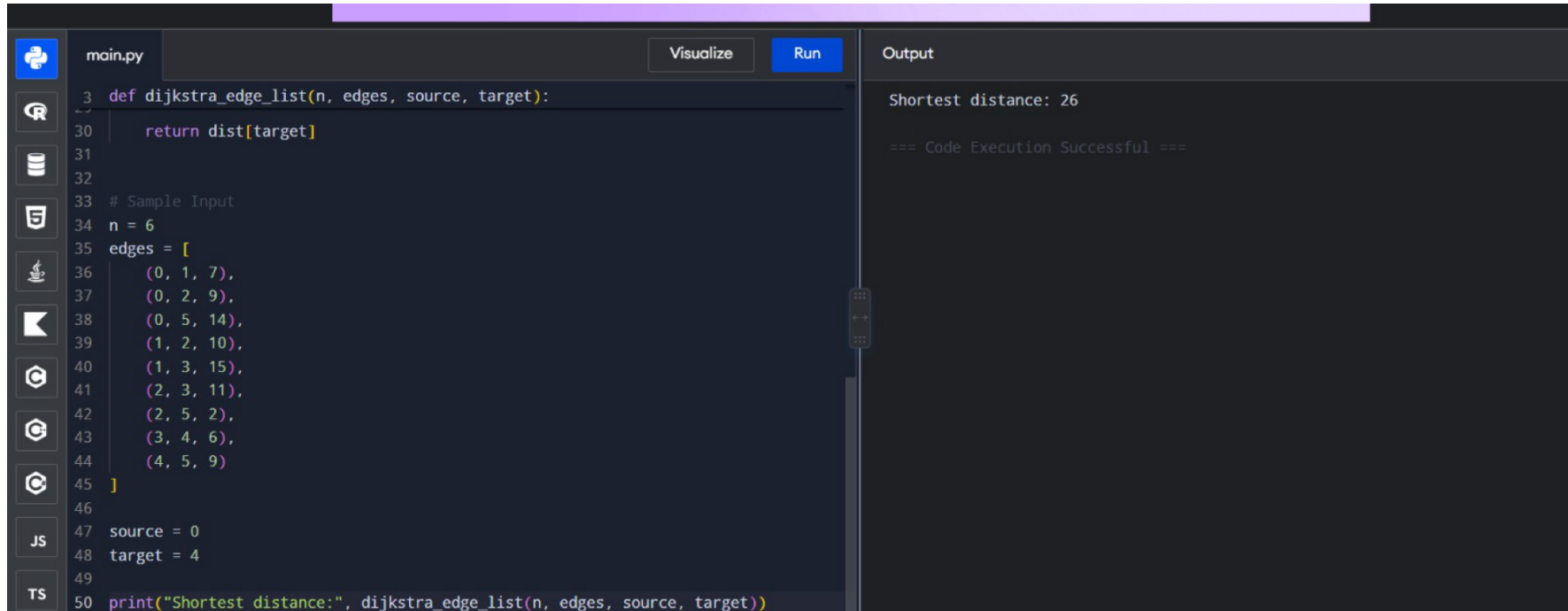
```
1 def dijkstra_matrix(graph, source):
9     for _ in range(n):
20         for v in range(n):
21             if graph[u][v] != float('inf') and not visited[v]:
26
27         return dist
28
29
30 # Sample Input
31 INF = float('inf')
32
33 graph = [
34     [0, 10, 3, INF, INF],
35     [INF, 0, 1, 2, INF],
36     [INF, 4, 0, 8, 2],
37     [INF, INF, INF, 0, 7],
38     [INF, INF, INF, 9, 0]
39 ]
40
41 source = 0
42
43 print("Shortest distances:", dijkstra_matrix(graph, source))
```

Output:

```
Shortest distances: [0, 7, 3, 9, 5]

=== Code Execution Successful ===
```

5. Dijkstra's Algorithm – Adjacency Matrix



The screenshot displays a Jupyter Notebook interface with a dark theme. The left sidebar contains icons for various tools: a Python logo, a Jupyter logo, a database icon, a file explorer icon, a terminal icon, a search icon, and a settings icon. Below these are tabs for 'JS' and 'TS'. The main area is divided into two sections: a code editor on the left and an output area on the right. The code editor shows a file named 'main.py' with the following Python code:

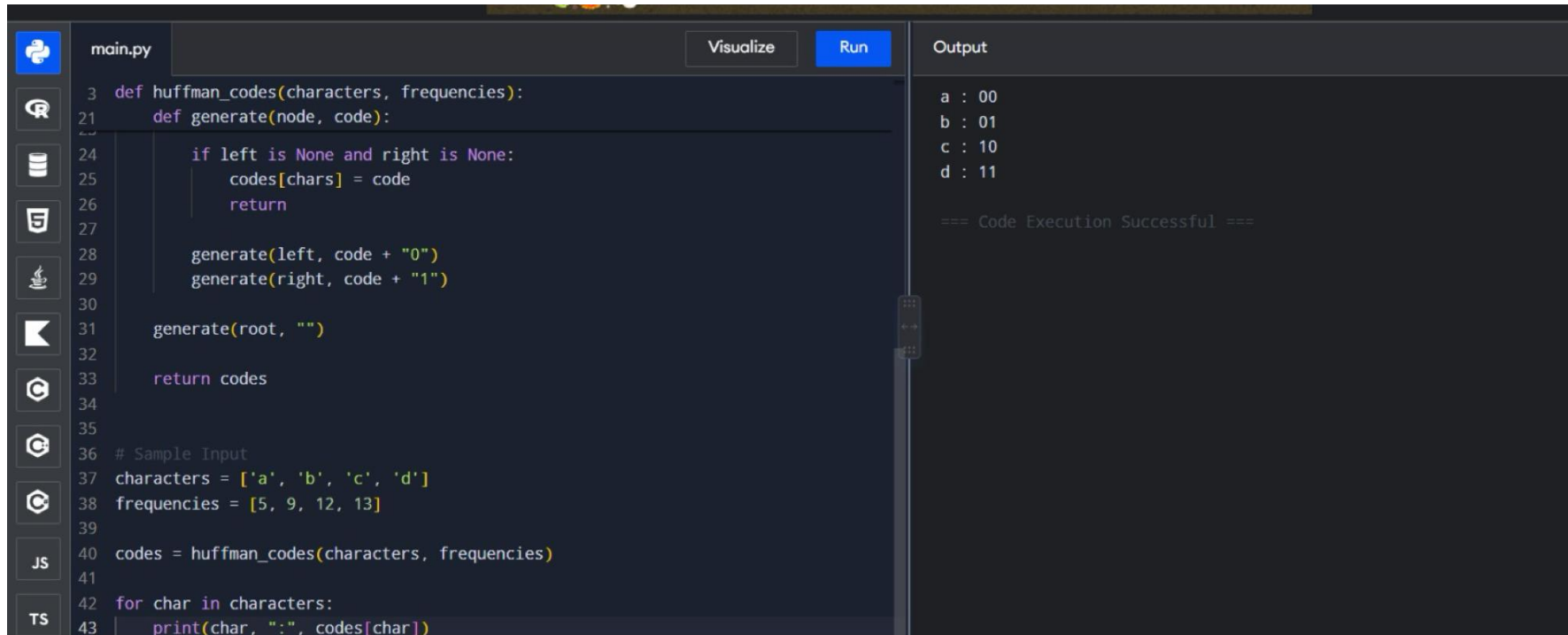
```
3 def dijkstra_edge_list(n, edges, source, target):
30     return dist[target]
31
32
33 # Sample Input
34 n = 6
35 edges = [
36     (0, 1, 7),
37     (0, 2, 9),
38     (0, 5, 14),
39     (1, 2, 10),
40     (1, 3, 15),
41     (2, 3, 11),
42     (2, 5, 2),
43     (3, 4, 6),
44     (4, 5, 9)
45 ]
46
47 source = 0
48 target = 4
49
50 print("Shortest distance:", dijkstra_edge_list(n, edges, source, target))
```

The output area on the right shows the result of the code execution:

```
Shortest distance: 26

=== Code Execution Successful ===
```

6. Dijkstra's Algorithm – Edge List



The image shows a code editor interface with a dark theme. The editor is displaying a Python file named `main.py`. The code implements a Huffman coding algorithm. It defines a `huffman_codes` function that takes `characters` and `frequencies` as input. Inside, it defines a `generate` function that recursively builds the Huffman tree. The `generate` function assigns binary codes to each character based on their frequency. The `huffman_codes` function then uses these codes to generate the final Huffman codes for each character. The code also includes sample input data and prints the resulting codes.

```
3 def huffman_codes(characters, frequencies):
21     def generate(node, code):
24         if left is None and right is None:
25             codes[chars] = code
26             return
27
28             generate(left, code + "0")
29             generate(right, code + "1")
30
31     generate(root, "")
32
33     return codes
34
35
36 # Sample Input
37 characters = ['a', 'b', 'c', 'd']
38 frequencies = [5, 9, 12, 13]
39
40 codes = huffman_codes(characters, frequencies)
41
42 for char in characters:
43     print(char, ":", codes[char])
```

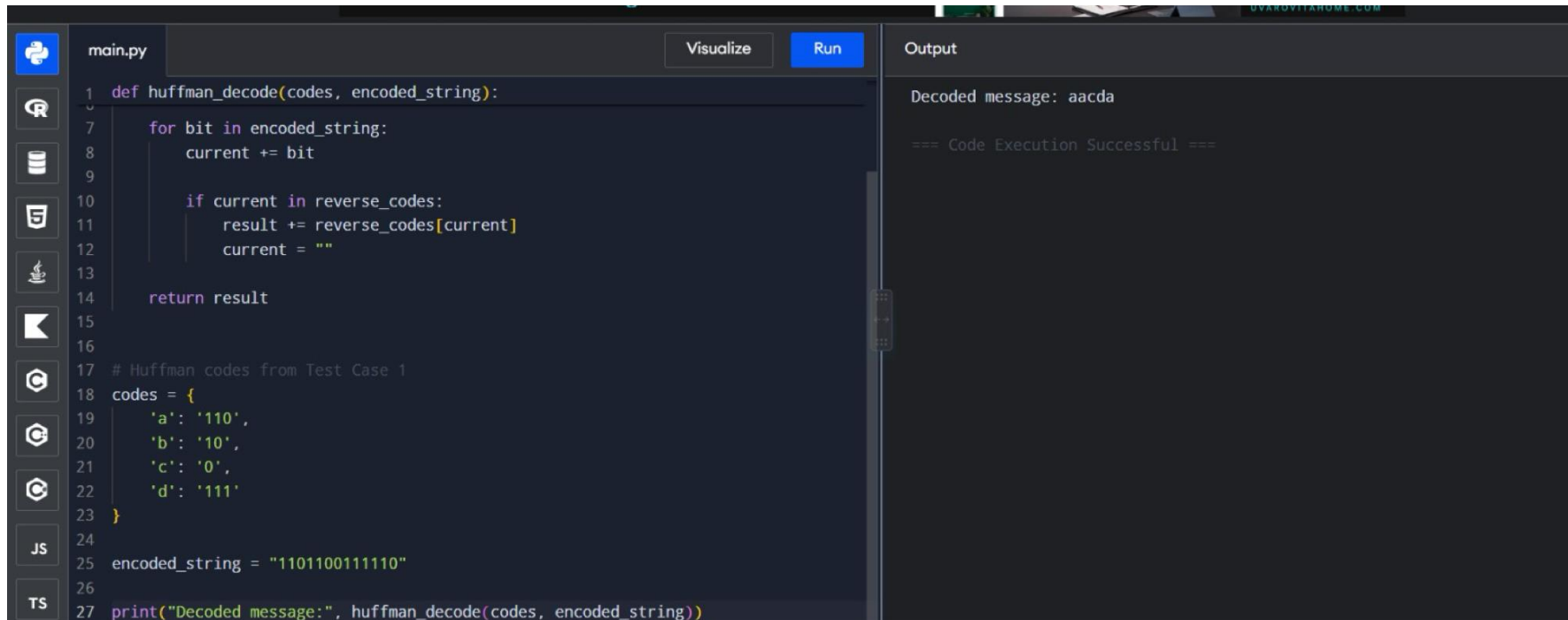
The output of the code execution is displayed on the right side of the editor. It shows the Huffman codes for each character: 'a' has code '00', 'b' has code '01', 'c' has code '10', and 'd' has code '11'. Below the codes, it states "=== Code Execution Successful ===".

Output

```
a : 00
b : 01
c : 10
d : 11

=== Code Execution Successful ===
```

7. Huffman Tree – Generate Codes



The image shows a code editor interface with a dark theme. On the left, there is a sidebar with icons for Python, Jupyter Notebook, and other languages. The main editor area displays a Python file named `main.py`. The code defines a `huffman_decode` function that takes `codes` and `encoded_string` as arguments. It iterates through the `encoded_string`, building up a `current` string of bits. When `current` is found in the `reverse_codes` dictionary, it appends the corresponding character to the `result` and resets `current`. The code also includes a test case with a dictionary of Huffman codes and an encoded string. The output panel on the right shows the decoded message as `aacda` and a success message.

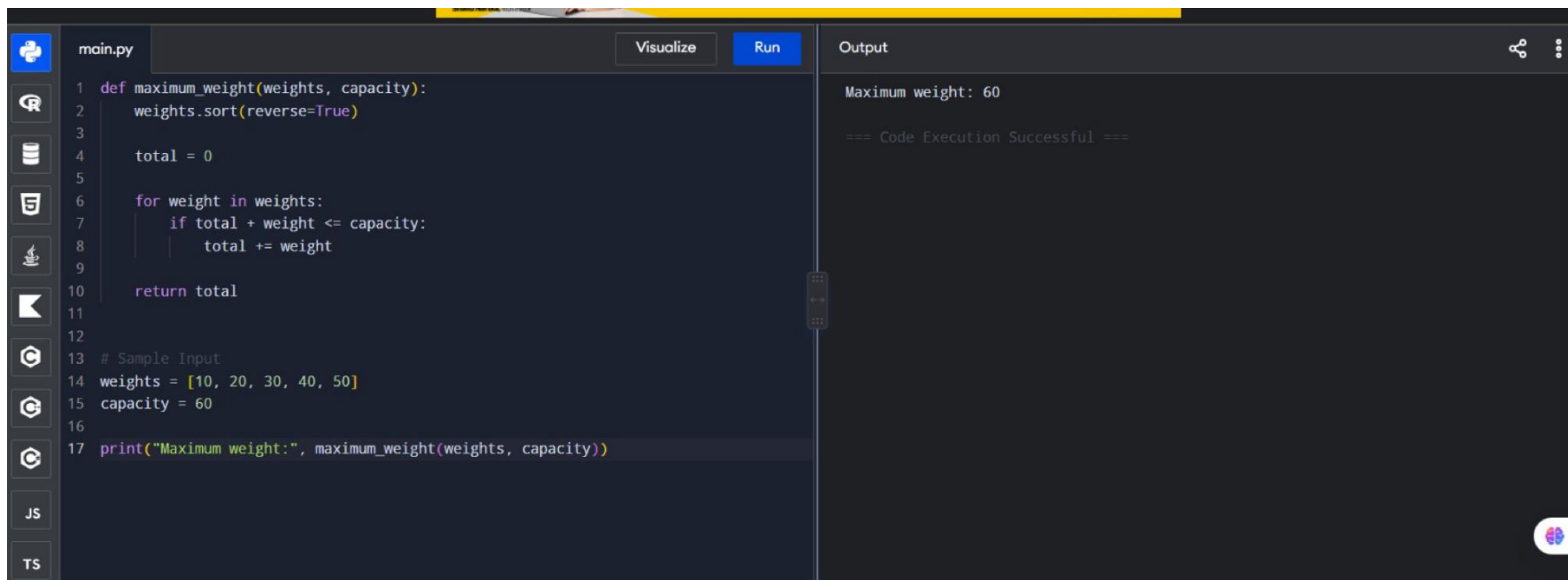
```
1 def huffman_decode(codes, encoded_string):
2
3     reverse_codes = {}
4     for char, code in codes.items():
5         reverse_codes[code] = char
6
7     for bit in encoded_string:
8         current += bit
9
10        if current in reverse_codes:
11            result += reverse_codes[current]
12            current = ""
13
14    return result
15
16
17 # Huffman codes from Test Case 1
18 codes = {
19     'a': '110',
20     'b': '10',
21     'c': '0',
22     'd': '111'
23 }
24
25 encoded_string = "1101100111110"
26
27 print("Decoded message:", huffman_decode(codes, encoded_string))
```

Output

Decoded message: aacda

=== Code Execution Successful ===

8. Huffman Decoding



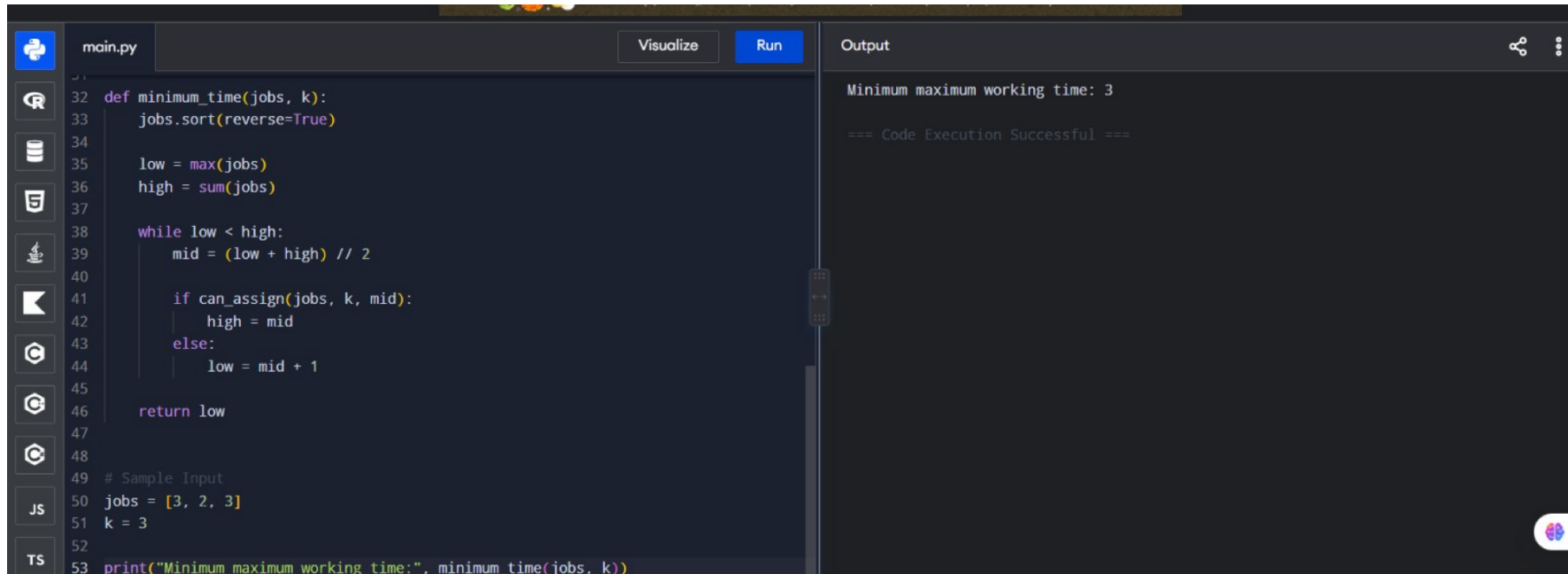
The image shows a code editor interface with a dark theme. On the left, a sidebar contains icons for file explorer, search, and other tools. The main editor area displays a Python file named 'main.py'. The code defines a function 'maximum_weight' that takes a list of weights and a capacity, sorts the weights in descending order, and iteratively adds them to a total until the capacity is reached. Below the function, sample input is provided: weights = [10, 20, 30, 40, 50] and capacity = 60. The function is then called with these inputs, and the result is printed. On the right, the 'Output' panel shows the result: 'Maximum weight: 60' and a success message '=== Code Execution Successful ==='. The editor also features 'Visualize' and 'Run' buttons at the top.

```
1 def maximum_weight(weights, capacity):
2     weights.sort(reverse=True)
3
4     total = 0
5
6     for weight in weights:
7         if total + weight <= capacity:
8             total += weight
9
10    return total
11
12
13 # Sample Input
14 weights = [10, 20, 30, 40, 50]
15 capacity = 60
16
17 print("Maximum weight:", maximum_weight(weights, capacity))
```

Maximum weight: 60

=== Code Execution Successful ===

9. Maximum Weight Using Greedy Approach



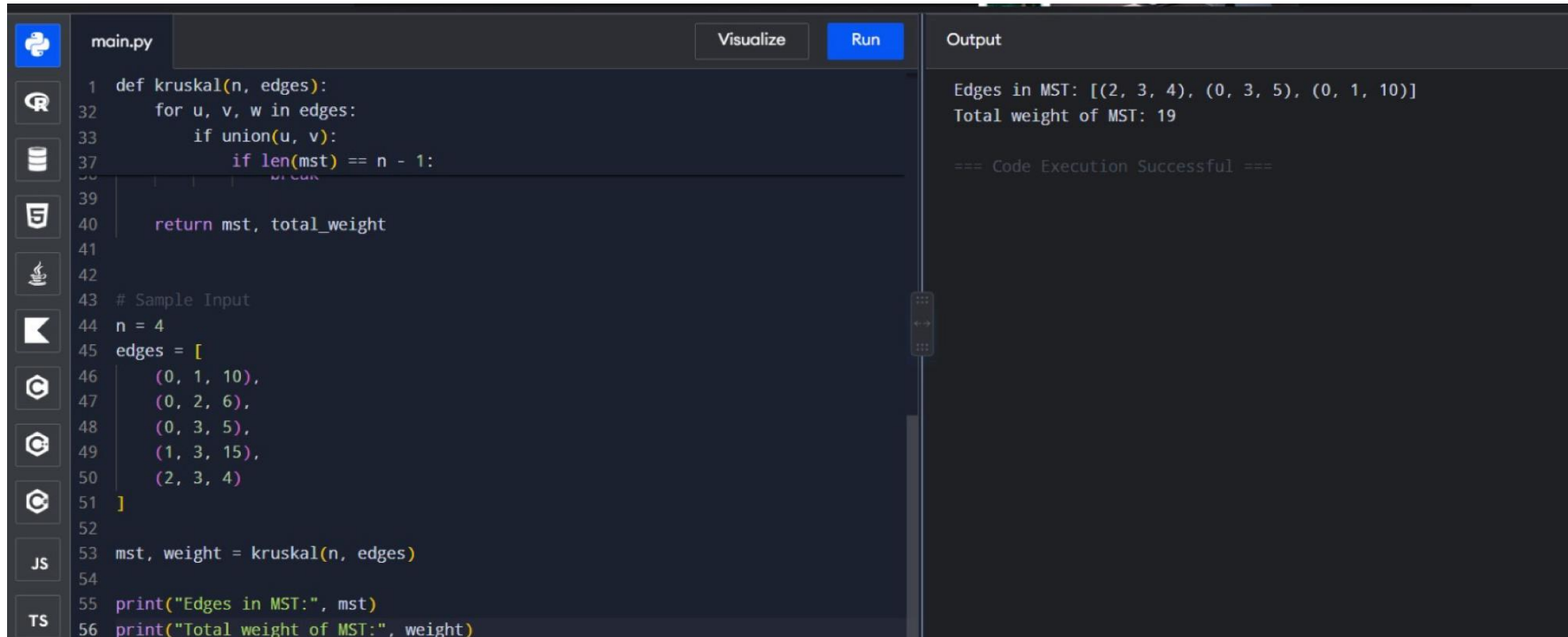
The image shows a code editor interface with a file named `main.py`. The code implements a binary search algorithm to find the minimum maximum working time for k workers given a list of jobs. The `minimum_time` function sorts jobs in descending order and uses a binary search between the maximum job time and the sum of all jobs. The `can_assign` function checks if k workers can complete the jobs within a given time limit. The sample input is `jobs = [3, 2, 3]` and `k = 3`, resulting in a minimum maximum working time of 3.

```
32 def minimum_time(jobs, k):
33     jobs.sort(reverse=True)
34
35     low = max(jobs)
36     high = sum(jobs)
37
38     while low < high:
39         mid = (low + high) // 2
40
41         if can_assign(jobs, k, mid):
42             high = mid
43         else:
44             low = mid + 1
45
46     return low
47
48
49 # Sample Input
50 jobs = [3, 2, 3]
51 k = 3
52
53 print("Minimum maximum working time:", minimum_time(jobs, k))
```

Output

```
Minimum maximum working time: 3
=== Code Execution Successful ===
```

10. Minimum Containers Using Greedy Approach



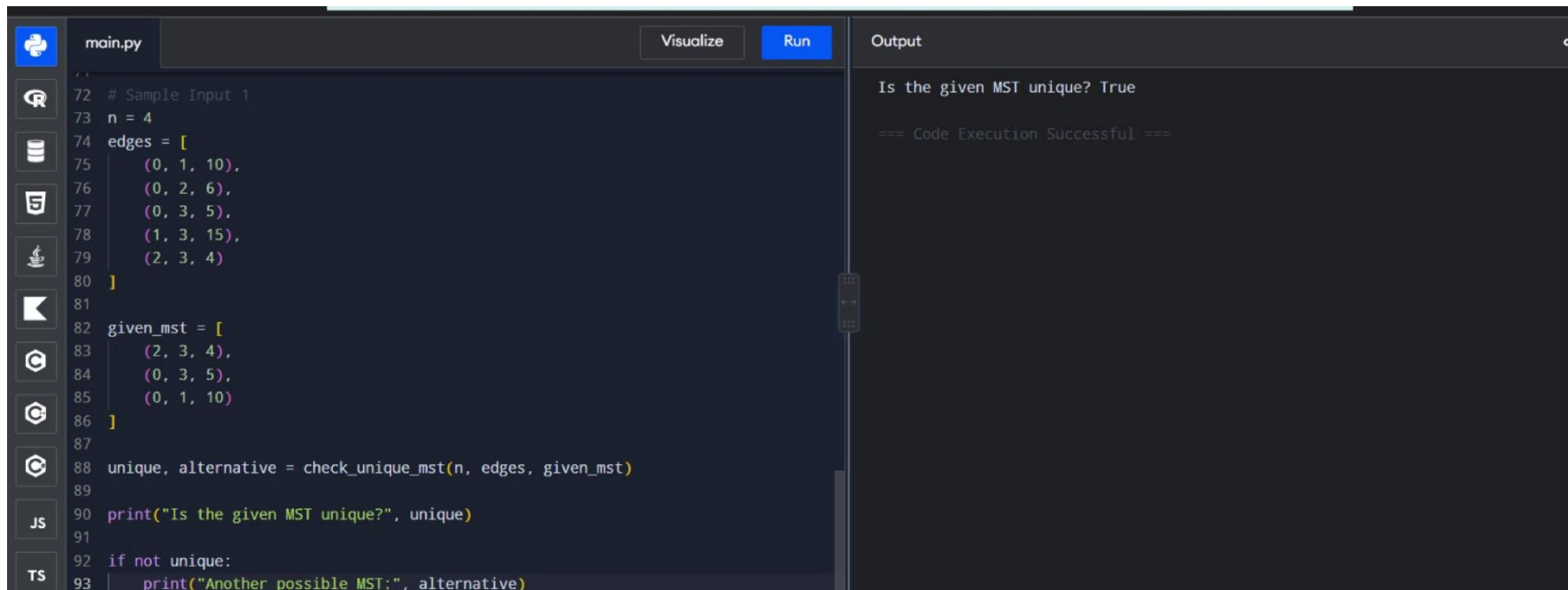
```
main.py Visualize Run
1 def kruskal(n, edges):
2     for u, v, w in edges:
3         if union(u, v):
4             if len(mst) == n - 1:
5                 break
6     return mst, total_weight
7
8 # Sample Input
9 n = 4
10 edges = [
11     (0, 1, 10),
12     (0, 2, 6),
13     (0, 3, 5),
14     (1, 3, 15),
15     (2, 3, 4)
16 ]
17
18 mst, weight = kruskal(n, edges)
19
20 print("Edges in MST:", mst)
21 print("Total weight of MST:", weight)
```

Output

Edges in MST: [(2, 3, 4), (0, 3, 5), (0, 1, 10)]
Total weight of MST: 19

=== Code Execution Successful ===

11. Kruskal's Algorithm – Minimum Spanning Tree



The image shows a code editor interface with a dark theme. On the left, there is a sidebar with icons for Python, JavaScript, and TypeScript. The main editor area displays a Python file named `main.py`. The code defines a sample input for Kruskal's Algorithm, including the number of nodes `n`, a list of edges with their weights, and a given Minimum Spanning Tree (`given_mst`). The code then calls a function `check_unique_mst` to verify if the given MST is unique. The output panel on the right shows the result of the execution.

```
main.py
72 # Sample Input 1
73 n = 4
74 edges = [
75     (0, 1, 10),
76     (0, 2, 6),
77     (0, 3, 5),
78     (1, 3, 15),
79     (2, 3, 4)
80 ]
81
82 given_mst = [
83     (2, 3, 4),
84     (0, 3, 5),
85     (0, 1, 10)
86 ]
87
88 unique, alternative = check_unique_mst(n, edges, given_mst)
89
90 print("Is the given MST unique?", unique)
91
92 if not unique:
93     print("Another possible MST:", alternative)
```

Output

```
Is the given MST unique? True

=== Code Execution Successful ===
```